

电子科技大学

实验报告

学生姓名：吕乐彬

学号：2023080904007

指导教师：李忻洋

一、实验项目名称： 实验4-1.在DLL导出表中查找函数地址、实验4-2.Dll导入表结构的验证、实验4-3.EPO之指令Patch

二、实验目的：

本次实验围绕 Windows PE 文件中的 DLL 导出表、DLL 导入表以及 EPO 指令 Patch 技术展开，目的是在静态分析和动态调试结合的过程中，理解 PE 文件中函数查找、导入解析和执行流修改的基本机制。

首先，通过分析 `user32.dll` 的导出表，掌握从数据目录定位导出表、从 RVA 换算文件偏移、读取 `IMAGE_EXPORT_DIRECTORY` 表头字段，并通过函数名指针表、序号表和函数地址表查找指定导出函数入口地址的方法。该部分实验重点在于理解 DLL 导出函数并不是直接按函数名保存地址，而是通过多张表共同完成“函数名到函数入口 RVA”的映射。

其次，通过分析 `Control.exe` 的导入表，掌握 `IMAGE_IMPORT_DESCRIPTOR`、`INT`、`IAT` 和 `Hint/Name` 结构之间的关系。通过比较程序加载前后的 IAT 内容，验证加载前 IAT 保存的是指向函数名结构的 RVA，而程序加载后 IAT 会被系统加载器改写为真实 API 函数入口地址。

最后，通过 EPO 指令 Patch 实验，理解在不直接修改 `AddressOfEntryPoint` 的情况下，如何利用程序原有执行路径中的 `CALL` 指令引入寄生代码，并在寄生代码执行完成后跳回原执行流程。该部分实验重点在于掌握代码空洞定位、节属性修改、相对跳转偏移计算、寄生代码写入和 `x32dbg` 动态验证方法。

三、实验原理：

PE 文件由 DOS 头、NT 头、可选映像头、节表和各节数据组成。程序在磁盘文

件中的偏移地址与加载到内存后的 RVA 并不完全相同，因此在分析导出表、导入表和指令位置时，需要频繁进行 RVA 与文件偏移之间的换算。若目标 RVA 位于某一节内，其文件偏移可按以下方式计算：

$$\text{文件偏移} = \text{RVA} - \text{节 RVA} + \text{PointerToRawData}$$

该公式是本实验中定位导出表、导入表、函数名字符串、IAT 表项和寄生代码位置的基础。

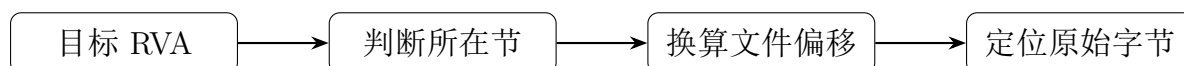


图 1 RVA 与文件偏移换算的基本分析流程

DLL 导出表用于描述 DLL 对外提供的函数。导出表的入口由可选映像头数据目录中的 `EXPORT Table` 指出。导出表表头中包含函数地址表 RVA、函数名指针表 RVA、序号表 RVA、导出序号基值以及函数数量等字段。通过函数名查找函数入口地址时，需要先在函数名指针表中找到函数名字符串，再通过同位置的序号表项得到函数地址表索引，最后在函数地址表中读取函数入口 RVA。其逻辑关系如下：

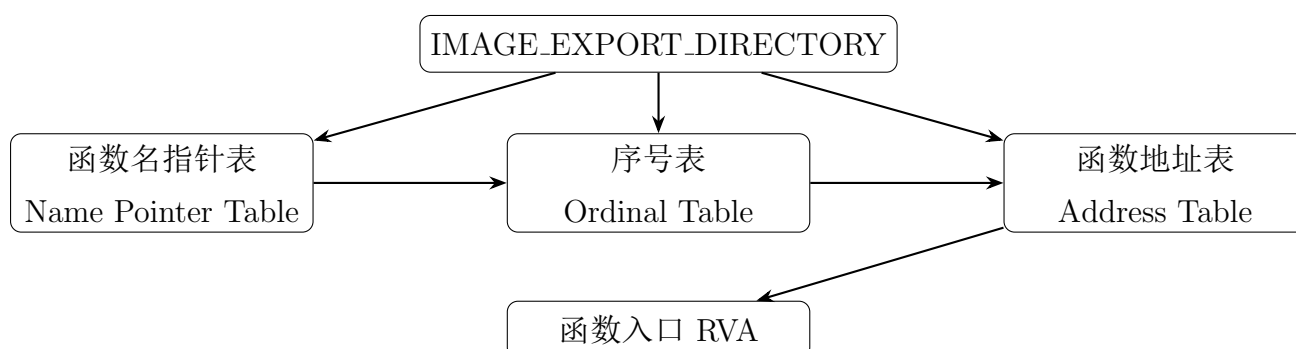


图 2 DLL 导出表中函数名到函数入口地址的映射关系

DLL 导入表用于描述当前 EXE 依赖哪些 DLL 以及需要使用其中哪些函数。每个 `IMAGE_IMPORT_DESCRIPTOR` 对应一个 DLL，其中 `OriginalFirstThunk` 或 `Import Name Table` 指向 INT，`FirstThunk` 或 `Import Address Table` 指向 IAT。加载前，INT 与 IAT 通常都保存指向 `IMAGE_IMPORT_BY_NAME` 的 RVA，也就是指向“Hint + 函数名字符串”的结构；加载后，系统加载器会根据 DLL 导出表解析真实函数地址，并将真实地址写入 IAT。

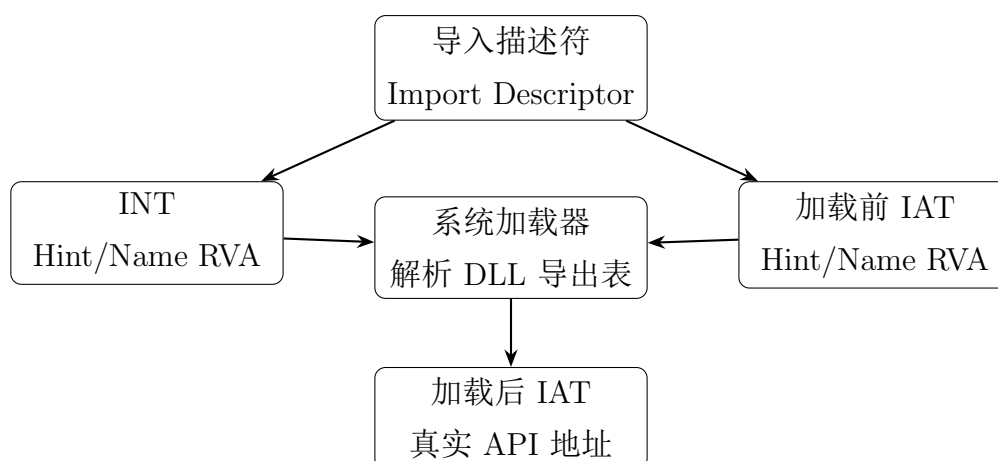


图 3 导入表加载前后 IAT 内容变化

EPO 指令 Patch 的核心思想是“不直接改入口点，而是改执行路径”。程序启动后仍从原入口点开始执行，但当执行到被选中的原有 CALL 或 JMP 指令时，程序会先跳转到寄生代码。寄生代码执行完成后，再通过尾部跳转回原来的 API 调用链或原程序指令位置，从而使原程序能够继续运行。本实验中选择入口点附近的相对 CALL 指令作为 Patch 点，将其改为调用 .text 节空洞中的寄生代码，再由寄生代码跳回 GetModuleHandleA 的导入跳板。

四、实验环境（设备、元器件）：

本实验在 Windows 系统环境下完成，实验对象包括 user32.dll 与 Control.exe。其中，user32.dll 用于 DLL 导出表分析，Control.exe 用于 DLL 导入表验证和 EPO 指令 Patch 实验。

实验过程中主要使用 PEView、ImHex 和 x32dbg 三类工具。PEView 用于解析 PE 文件结构，查看数据目录、节表、导出表、导入描述符、INT 和 IAT 等结构化字段；ImHex 用于直接查看和修改 PE 文件原始十六进制字节，完成 RVA 与文件偏移换算后的定位、寄生代码写入、节字段修改和指令 Patch；x32dbg 用于动态调试修改后的程序，观察程序加载基址、入口点指令、寄生代码执行过程以及 IAT 加载后的变化。

实验文件与截图统一放置在实验目录中，报告中的截图文件统一保存在 img 文件夹下。实验过程中使用的主要文件包括：

user32.dll, Control.exe

使用的主要工具包括：

PEView, ImHex, x32dbg

五、实验步骤：

1

1.1 在 DLL 导出表中查找函数地址

1.1.1 实验目标与查找思路

本实验以 `user32.dll` 为分析对象，目标是在 DLL 的导出表中查找指定导出函数的入口地址。DLL 作为 PE 文件的一种，同样包含 DOS 头、PE 头、可选映像头和节表等结构。导出函数的信息并不是直接散乱存放在文件中，而是由数据目录中的 `EXPORT Table` 指向导出表结构，再由导出表中的函数地址表、函数名指针表和序号表共同完成函数名到函数入口地址的映射。

本实验按照以下顺序进行分析：首先在可选映像头的数据目录中找到导出表入口 RVA；然后根据节表判断导出表位于哪个节，并将 RVA 换算为文件偏移；随后观察 `IMAGE_EXPORT_DIRECTORY` 表头中的关键字段；最后通过函数名指针表、序号表和函数地址表，查找学号后两位对应导出函数的入口地址，并在文件中定位该函数入口处的机器码。

1.1.2 定位导出表并观察表头字段

使用 PEView 打开实验提供的 `user32.dll`，在 `IMAGE_OPTIONAL_HEADER` 的数据目录中找到 `EXPORT Table` 项。该项给出了导出表的入口 RVA 和导出表大小。本实验中导出表信息如下：

`Export Table RVA = 0x00003900`

`Export Table Size = 0x00004BA9`

随后查看 `.text` 节表字段。由 PEView 可知，`.text` 节的关键字段为：

`VirtualSize = 0x0005F283, RVA = 0x00001000, PointerToRawData =
0x00000400`

因此，`.text` 节在内存中的 RVA 范围为：

$0x1000 < \text{RVA} < 0x1000 + 0x5F283 = 0x60283$

由于：

$0x1000 < 0x3900 < 0x60283$

所以可以判断导出表位于 `.text` 节中。根据 RVA 到文件偏移的换算公式：

$$\text{文件偏移} = \text{RVA} - \text{节 RVA} + \text{PointerToRawData}$$

可得导出表表头的文件偏移为：

$$0x3900 - 0x1000 + 0x0400 = 0x2D00$$

因此，IMAGE_EXPORT_DIRECTORY 的文件偏移为 0x2D00。

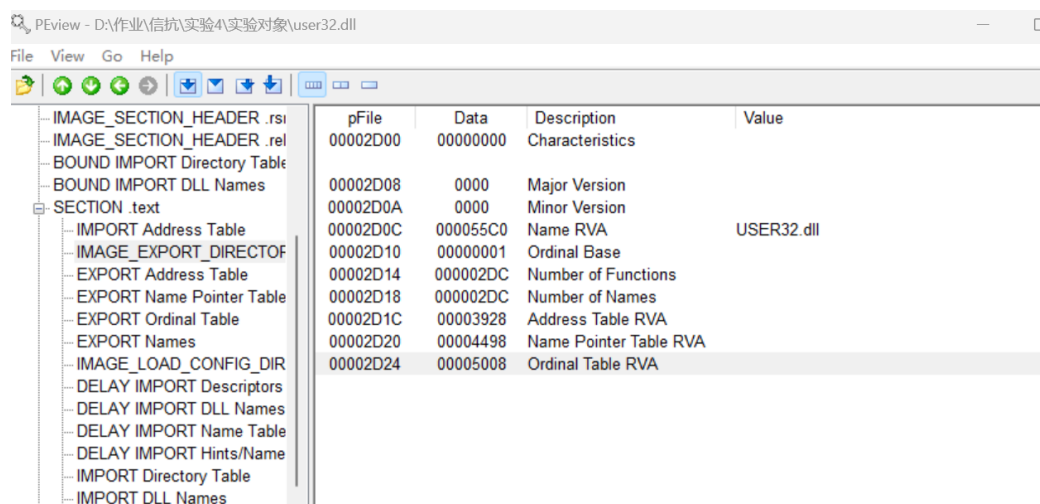


图 4 查看 IMAGE_EXPORT_DIRECTORY 表头字段

如图 4 所示，导出表表头中的关键字段如下：

$$\text{Name RVA} = 0x000055C0$$

$$\text{Ordinal Base} = 0x00000001$$

$$\text{Number of Functions} = 0x000002DC$$

$$\text{Number of Names} = 0x000002DC$$

$$\text{Address Table RVA} = 0x00003928$$

$$\text{Name Pointer Table RVA} = 0x00004498$$

$$\text{Ordinal Table RVA} = 0x00005008$$

其中，Address Table RVA 指向函数地址表，表中存放导出函数入口 RVA；Name Pointer Table RVA 指向函数名指针表，表中存放函数名字符串的 RVA；Ordinal Table RVA 指向序号表，表中存放函数名对应到函数地址表的索引。

1.1.3 使用编辑器验证导出表表头内容

为了验证 PEView 解析结果与文件中的原始字节一致，使用 ImHex 打开 user32.dll，跳转到前面计算得到的导出表文件偏移 0x2D00。

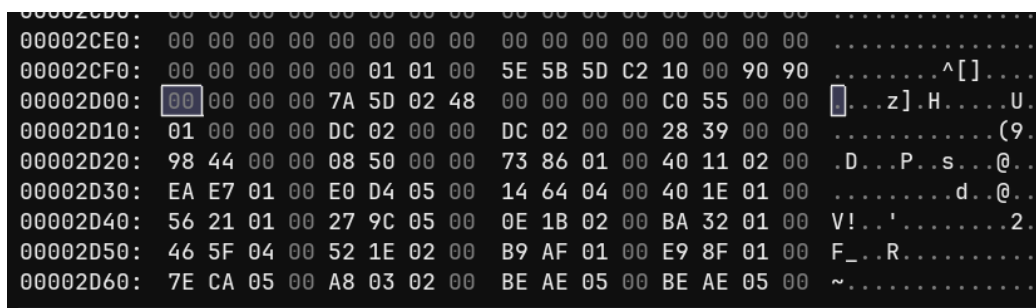


图 5 在 ImHex 中验证导出表表头原始字节

如图 5 所示，文件偏移 0x2D00 处的原始字节与 PEView 中解析出的导出表字段能够对应。由于 PE 文件采用小端序存储多字节整数，因此各字段在文件中的字节顺序与数值显示顺序相反。例如：

C0 55 00 00 -> Name RVA = 0x000055C0

01 00 00 00 -> Ordinal Base = 0x00000001

DC 02 00 00 -> Number of Functions = 0x000002DC

DC 02 00 00 -> Number of Names = 0x000002DC

28 39 00 00 -> Address Table RVA = 0x00003928

98 44 00 00 -> Name Pointer Table RVA = 0x00004498

08 50 00 00 -> Ordinal Table RVA = 0x00005008

由此可以确认，导出表表头的文件偏移 0x2D00 计算正确，且 PEView 对导出表字段的解析结果与文件中的实际字节一致。

1.1.4 通过函数名指针表验证函数名字符串

接下来查看函数名指针表。根据导出表表头，函数名指针表的 RVA 为 0x4498。由于该表也位于 .text 节中，其文件偏移为：

$$0x4498 - 0x1000 + 0x0400 = 0x3898$$

在 PEView 中打开 EXPORT Name Pointer Table，观察函数名指针表内容。

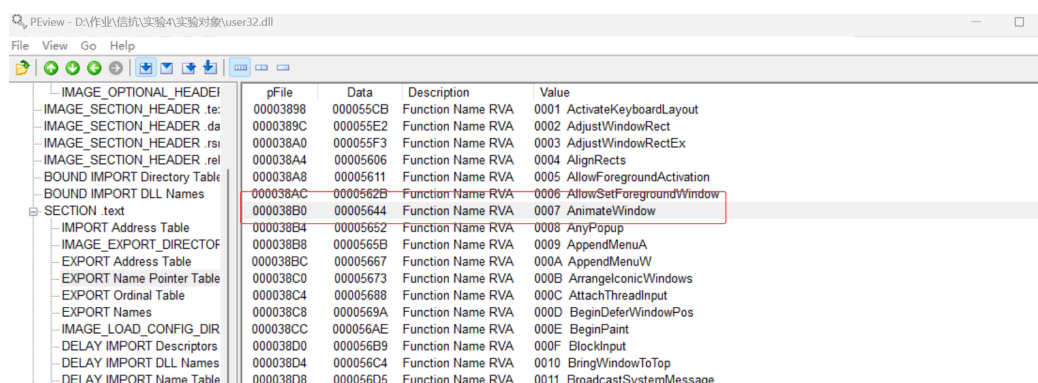


图 6 查看 EXPORT Name Pointer Table 并定位 AnimateWindow

如图 6 所示，学号末位为 7，对应的函数名指针表项显示为：

Function Name RVA = 0x00005644

Function Name = AnimateWindow

这说明函数名指针表中存放的并不是函数入口地址，而是函数名字符串的 RVA。将该 RVA 换算为文件偏移：

$$0x5644 - 0x1000 + 0x0400 = 0x4A44$$

随后在 ImHex 中跳转到文件偏移 0x4A44，验证该处是否确实为函数名字符串。

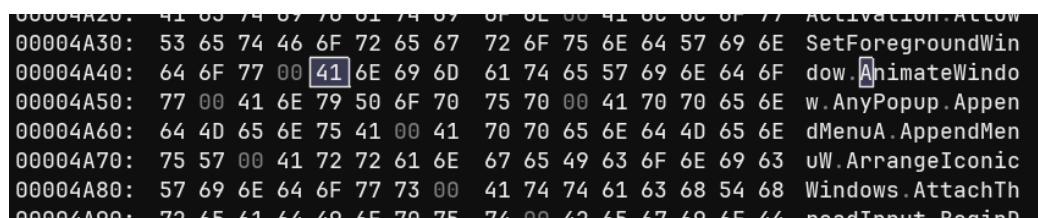
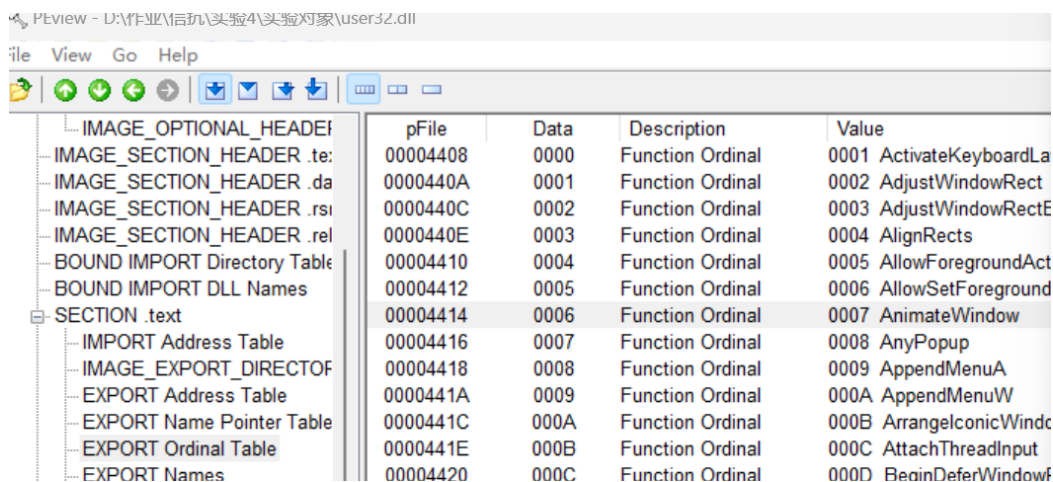


图 7 在 ImHex 中验证 0x5644 指向 AnimateWindow 字符串

如图 7 所示，文件偏移 0x4A44 处可以看到字符串 AnimateWindow，其前后还连续存放了其他导出函数名，如 AllowSetForegroundWindow、AnyPopup 等。这说明函数名指针表中的 Function Name RVA 确实指向文件中的函数名字符串。

1.1.5 通过序号表确定函数地址表索引

函数名指针表只能定位函数名字符串，不能直接得到函数入口地址。要从函数名找到函数地址，还需要通过序号表获得该函数在函数地址表中的索引。



	pFile	Data	Description	Value
IMAGE_OPTIONAL_HEADER				
IMAGE_SECTION_HEADER .text	00004408	0000	Function Ordinal	0001 ActivateKeyboardLa
IMAGE_SECTION_HEADER .data	0000440A	0001	Function Ordinal	0002 AdjustWindowRect
IMAGE_SECTION_HEADER .rsi	0000440C	0002	Function Ordinal	0003 AdjustWindowRectE
IMAGE_SECTION_HEADER .rel	0000440E	0003	Function Ordinal	0004 AlignRects
BOUND_IMPORT Directory Table	00004410	0004	Function Ordinal	0005 AllowForegroundAct
BOUND_IMPORT DLL Names	00004412	0005	Function Ordinal	0006 AllowSetForeground
SECTION .text	00004414	0006	Function Ordinal	0007 AnimateWindow
IMPORT Address Table	00004416	0007	Function Ordinal	0008 AnyPopup
IMAGE_EXPORT_DIRECTORY	00004418	0008	Function Ordinal	0009 AppendMenuA
EXPORT Address Table	0000441A	0009	Function Ordinal	000A AppendMenuW
EXPORT Name Pointer Table	0000441C	000A	Function Ordinal	000B ArrangeIconicWindo
EXPORT Ordinal Table	0000441E	000B	Function Ordinal	000C AttachThreadInput
EXPORT Names	00004420	000C	Function Ordinal	000D BeginDeferWindowf

图 8 查看 EXPORT Ordinal Table 并确定函数地址表索引

如图 8 所示, AnimateWindow 对应的序号表项为:

Function Ordinal Data = 0x0006

由于该值表示函数地址表中的索引, 因此:

AnimateWindow 在 EXPORT Address Table 中的索引 = 0x0006

同时, PEView 的 Value 栏中显示 0007 AnimateWindow, 这是因为导出表中的 Ordinal Base = 1, 因此导出序号为:

Ordinal = Ordinal Base + Index = 1 + 6 = 7

这说明学号后两位 07 对应的导出函数为 AnimateWindow。

1.1.6 查找函数入口 RVA 并定位函数入口

根据导出表表头, 函数地址表的 RVA 为 0x3928。函数地址表中每一项占 4 字节, AnimateWindow 对应的索引为 0x0006, 因此该函数在函数地址表中的表项 RVA 为:

$0x3928 + 6 * 4 = 0x3940$

换算为文件偏移:

$0x3940 - 0x1000 + 0x0400 = 0x2D40$

在前面对导出表原始字节的验证图中, 文件偏移 0x2D40 处的 4 字节为:

56 21 01 00

按小端序还原后得到：

AnimateWindow 函数入口 RVA = 0x00012156

由于该 RVA 仍位于 .text 节中，因此其文件偏移为：

$0x12156 - 0x1000 + 0x0400 = 0x11556$

最后在 ImHex 中跳转到文件偏移 0x11556，观察该函数入口处的机器码。

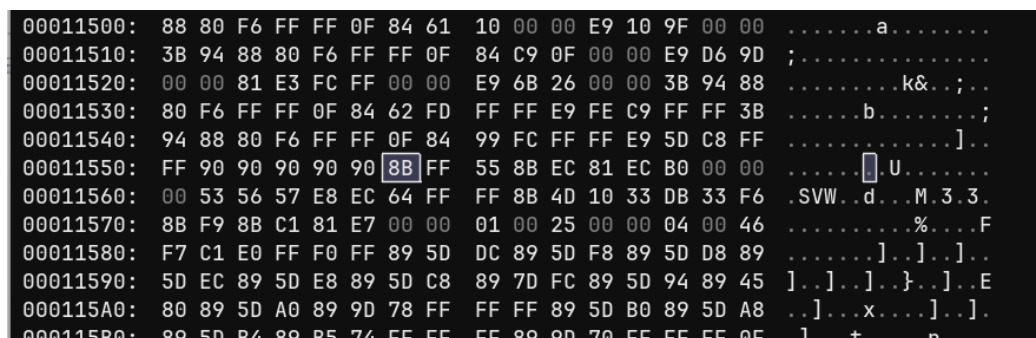


图 9 在 ImHex 中观察 AnimateWindow 函数入口机器码

如图 9 所示，在文件偏移 0x11556 处可以看到函数入口处的机器码：

8B FF 55 8B EC ...

其中 8B FF 55 8B EC 是常见的函数入口序列，对应反汇编形式通常为：

mov edi, edi; push ebp; mov ebp, esp

由此可以确认，通过导出表查找到的 AnimateWindow 函数入口 RVA 为 0x12156，对应文件偏移为 0x11556，查找过程正确。

1.2 DLL 导入表结构的验证

1.2.1 实验目标与验证思路

本实验以 Control.exe 为分析对象，验证 Windows PE 文件的导入表机制。导入表的作用是描述当前程序需要从哪些 DLL 中导入哪些 API 函数。程序在磁盘文件中并不直接保存被导入函数的真实内存地址，而是保存指向函数名结构的 RVA；当程序被系统加载后，加载器会根据导入表找到对应 DLL，再通过 DLL 的导出表查找函数入口地址，并将真实函数地址写入 IAT 中。

因此，本实验主要验证两件事：第一，加载前的 INT 和 IAT 中保存的是 Hint/Name RVA，也就是指向函数名结构的 RVA；第二，加载后内存中的 IAT 会被系统加载器改写为真实 API 函数入口地址。实验过程按照“定位导入表、观察导入描述符、比较加载前 INT/IAT、验证 Hint/Name 结构、观察加载后 IAT”的顺序展开。

1.2.2 定位导入表 RVA 与所在节

首先使用 PEView 打开 Control.exe，在 IMAGE_OPTIONAL_HEADER 的数据目录中查找 IMPORT Table 项。该项给出了导入表的入口 RVA 和大小。

-MS-DOS Stub Program	00000140	00000000	Loader Flags	
3-IMAGE_NT_HEADERS	00000144	00000010	Number of Data Directories	
Signature	00000148	00000000	RVA	EXPORT Table
IMAGE_FILE_HEADER	0000014C	00000000	Size	
IMAGE_OPTIONAL_HEADER	00000150	000020C4	RVA	IMPORT Table
-IMAGE_SECTION_HEADER .text	00000154	00000050	Size	
-IMAGE_SECTION_HEADER .rdata	00000158	00004000	RVA	RESOURCE Table
-IMAGE_SECTION_HEADER .data	0000015C	0000F078	Size	

图 10 在数据目录中查看 IMPORT Table 的 RVA 和 Size

如图 10 所示，Control.exe 的导入表信息为：

IMPORT Table RVA = 0x000020C4

IMPORT Table Size = 0x00000050

随后查看节表，判断导入表 RVA 0x20C4 落在哪个节中。

	pFile	Data	Description	Value
Control.exe				
IMAGE_DOS_HEADER	000001F0	2E 72 64 61	Name	.rdata
MS-DOS Stub Program	000001F4	74 61 00 00		
IMAGE_NT_HEADERS	000001F8	000002F6	Virtual Size	
Signature	000001FC	00002000	RVA	
IMAGE_FILE_HEADER	00000200	00000400	Size of Raw Data	
IMAGE_OPTIONAL_HEADER	00000204	00000A00	Pointer to Raw Data	
IMAGE_SECTION_HEADER .text	00000208	00000000	Pointer to Relocations	
IMAGE_SECTION_HEADER .rdata	0000020C	00000000	Pointer to Line Numbers	
IMAGE_SECTION_HEADER .data	00000210	0000	Number of Relocations	
IMAGE_SECTION_HEADER .rsrc	00000212	0000	Number of Line Numbers	
IMAGE_SECTION_HEADER .reloc	00000214	40000040	Characteristics	
SECTION .text		00000040		IMAGE_SCN_CNT_INITIALIZED_DATA
SECTION .rdata		40000000		IMAGE_SCN_MEM_READ
SECTION .rsrc				
SECTION .reloc				

图 11 查看 .rdata 节表字段并判断导入表所在节

如图 11 所示，.rdata 节的关键字段为：

VirtualSize = 0x000002F6, RVA = 0x00002000, PointerToRawData = 0x00000A00

因此，.rdata 节在内存中的 RVA 范围为：

$0x2000 < \text{RVA} < 0x2000 + 0x02F6 = 0x22F6$

由于：

$0x2000 < 0x20C4 < 0x22F6$

所以可以判断 IMPORT Table 位于 .rdata 节中。根据 RVA 到文件偏移的换算公式：

$$\text{文件偏移} = \text{RVA} - \text{节 RVA} + \text{PointerToRawData}$$

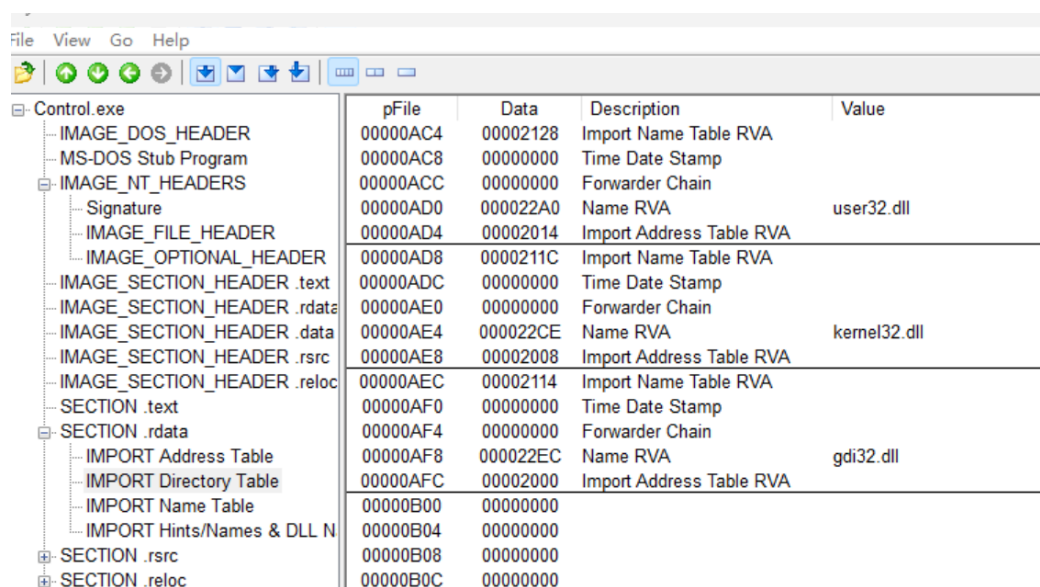
可得导入表的文件偏移为：

$$0x20C4 - 0x2000 + 0x0A00 = 0x0AC4$$

因此，IMPORT Directory Table 的文件偏移为 0x0AC4。

1.2.3 观察导入描述符结构

在 PEView 中展开 SECTION .rdata，打开 IMPORT Directory Table，观察每个导入描述符。每个 IMAGE_IMPORT_DESCRIPTOR 对应一个被导入的 DLL，其中 Import Name Table RVA 指向 INT，Name RVA 指向 DLL 名字字符串，Import Address Table RVA 指向 IAT。



pFile	Data	Description	Value
00000AC4	00002128	Import Name Table RVA	
00000AC8	00000000	Time Date Stamp	
00000ACC	00000000	Forwarder Chain	
00000AD0	000022A0	Name RVA	user32.dll
00000AD4	00002014	Import Address Table RVA	
00000AD8	0000211C	Import Name Table RVA	
00000ADC	00000000	Time Date Stamp	
00000AE0	00000000	Forwarder Chain	
00000AE4	000022CE	Name RVA	kernel32.dll
00000AE8	00002008	Import Address Table RVA	
00000AEC	00002114	Import Name Table RVA	
00000AF0	00000000	Time Date Stamp	
00000AF4	00000000	Forwarder Chain	
00000AF8	000022EC	Name RVA	gdi32.dll
00000AFC	00002000	Import Address Table RVA	
00000B00	00000000		
00000B04	00000000		
00000B08	00000000		
00000B0C	00000000		

图 12 查看 IMPORT Directory Table 中的导入描述符

如图 12 所示，当前程序导入了 user32.dll、kernel32.dll 和 gdi32.dll。其中，user32.dll 对应的导入描述符字段为：

$$\text{Import Name Table RVA} = 0x00002128$$

$$\text{Name RVA} = 0x000022A0$$

$$\text{Import Address Table RVA} = 0x00002014$$

kernel32.dll 对应的导入描述符字段为:

Import Name Table RVA = 0x0000211C

Name RVA = 0x000022CE

Import Address Table RVA = 0x00002008

gdi32.dll 对应的导入描述符字段为:

Import Name Table RVA = 0x00002114

Name RVA = 0x000022EC

Import Address Table RVA = 0x00002000

导入描述符表后方出现连续的 0x00000000, 表示导入描述符数组结束。

1.2.4 观察加载前的 INT 与 IAT

接下来分别查看 IMPORT Name Table 和 IMPORT Address Table。在程序加载前,二者中的表项都保存 Hint/Name RVA, 即指向 IMAGE_IMPORT_BY_NAME 结构的 RVA, 而不是 API 函数的真实入口地址。

Control.exe	pFile	Data	Description	Value
IMAGE_DOS_HEADER	00000B14	000022DC	Hint/Name RVA	004B DeleteObject
MS-DOS Stub Program	00000B18	00000000	End of Imports	gdi32.dll
IMAGE_NT_HEADERS	00000B1C	000022AC	Hint/Name RVA	009B ExitProcess
Signature	00000B20	000022BA	Hint/Name RVA	0134 GetModuleHandleA
IMAGE_FILE_HEADER	00000B24	00000000	End of Imports	kernel32.dll
IMAGE_OPTIONAL_HEADER	00000B28	000021C2	Hint/Name RVA	00FC GetDlgItemTextA
IMAGE_SECTION_HEADER.text	00000B2C	000021D4	Hint/Name RVA	0152 GetWindowLongA
IMAGE_SECTION_HEADER.rdata	00000B30	000021E6	Hint/Name RVA	0183 IsDlgButtonChecked
IMAGE_SECTION_HEADER.data	00000B34	000021FC	Hint/Name RVA	018D IsWindowVisible
IMAGE_SECTION_HEADER.rsrc	00000B38	0000220E	Hint/Name RVA	0192 LoadBitmapA
IMAGE_SECTION_HEADER.reloc	00000B3C	0000221C	Hint/Name RVA	0198 LoadIconA
SECTION.text	00000B40	000021B4	Hint/Name RVA	00FA GetDlgItem
SECTION.rdata	00000B44	0000223E	Hint/Name RVA	01FD SendMessageA
IMPORT Address Table	00000B48	0000224E	Hint/Name RVA	0212 SetDlgItemInt
IMPORT Directory Table	00000B4C	0000225E	Hint/Name RVA	0238 SetWindowLongA
IMPORT Name Table	00000B50	00002270	Hint/Name RVA	023B SetWindowPos
IMPORT Hints/Names & DLL N	00000B54	00002280	Hint/Name RVA	023D SetWindowTextA
SECTION.rsrc	00000B58	00002292	Hint/Name RVA	0248 ShowWindow
SECTION.reloc	00000B5C	000021A8	Hint/Name RVA	00B4 EndDialog
	00000B60	00002186	Hint/Name RVA	0090 DialogBoxParamA
	00000B64	00002198	Hint/Name RVA	00B2 EnableWindow
	00000B68	00002228	Hint/Name RVA	01F8 SendDlgItemMessageA
	00000B6C	00002174	Hint/Name RVA	0032 CheckDlgButton
	00000B70	00000000	End of Imports	user32.dll

(a) 加载前 IMPORT Name Table 中的 Hint/Name RVA

(b) 加载前 IMPORT Address Table 中的 Hint/Name RVA

图 13 加载前 INT 与 IAT 表项内容对比

如图 13 所示, 加载前 IMPORT Name Table 与 IMPORT Address Table 中的内容基本一致。以 user32.dll 中的 GetDlgItemTextA 为例:

IMPORT Name Table: Data = 0x000021C2, Value = GetDlgItemTextA

IMPORT Address Table: Data = 0x000021C2, Value = GetDlgItemTextA

这说明加载前 IAT 并没有保存 GetDlgItemTextA 的真实函数地址, 而是保存了一个指向函数名结构的 RVA, 即 0x21C2。

1.2.5 验证 Hint/Name RVA 指向函数名结构

为进一步确认 0x21C2 的含义，将其换算为文件偏移。由于该 RVA 位于 .rdata 节中，因此：

$$0x21C2 - 0x2000 + 0x0A00 = 0x0BC2$$

使用 ImHex 打开 Control.exe，跳转到文件偏移 0x0BC2，观察对应内容。

```

00000B90: 78 50 61 72 61 6D 41 00 B2 00 45 6E 61 62 6C 65 xParamA...Enable
00000BA0: 57 69 6E 64 6F 77 00 00 B4 00 45 6E 64 44 69 61 Window...EndDia
00000BB0: 6C 6F 67 00 FA 00 47 65 74 44 6C 67 49 74 65 6D log...GetDlgItem
00000BC0: 00 00 FC 00 47 65 74 44 6C 67 49 74 65 6D 54 65 ..GetDlgItemTe
00000BD0: 78 74 41 00 52 01 47 65 74 57 69 6E 64 6F 77 4C xtA.R.GetWindowL
00000BE0: 6F 6E 67 41 00 00 83 01 49 73 44 6C 67 42 75 74 ongA...IsDlgBut
00000BF0: 74 6F 6E 43 68 65 63 6B 65 64 00 00 8D 01 49 73 tonChecked...Is
00000C00: 57 69 6E 64 6F 77 56 69 73 69 62 6C 65 00 92 01 WindowVisible...
00000C10: 4C 6F 61 64 42 69 74 6D 61 70 41 00 98 01 4C 6F LoadBitmapA...Lo
00000C20: 61 64 49 63 6F 6E 41 00 F8 01 53 65 6E 64 44 6C adIconA...SendDL
00000C30: 67 49 74 65 6D 4D 65 73 73 61 67 65 41 00 FD 01 gItemMessageA...
00000C40: 53 65 6E 64 4D 65 73 73 61 67 65 41 00 00 12 02 SendMessageA...

```

图 14 在 ImHex 中验证 0x21C2 指向 GetDlgItemTextA 的 Hint/Name 结构

如图 14 所示，在文件偏移 0x0BC2 处可以看到：

```
FC 00 47 65 74 44 6C 67 49 74 65 6D 54 65 78 74 41 00
```

其中，前两个字节 FC 00 是 Hint 值，按小端序表示：

$$\text{Hint} = 0x00FC$$

后面的 ASCII 字符串为：

```
GetDlgItemTextA\0
```

这说明加载前 IAT 表项 0x21C2 确实指向 IMAGE_IMPORT_BY_NAME 结构，即“Hint + 函数名字符串”，而不是函数真实入口地址。

1.2.6 观察加载后的 IAT 内容

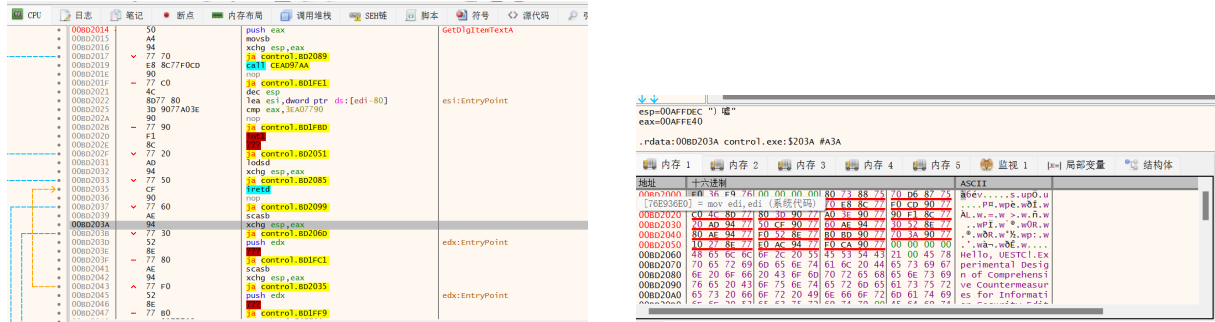
随后使用 x32dbg 打开 Control.exe。程序加载后，首先根据入口点地址计算实际加载基址。x32dbg 中显示入口点地址为 0x00BD1404，而原入口点 RVA 为 0x1404，因此实际加载基址为：

$$0x00BD1404 - 0x1404 = 0x00BD0000$$

前面已经确定 user32.dll 对应的 IAT RVA 为 0x2014，因此加载后 IAT 的内存地址为：

$$0x00BD0000 + 0x2014 = 0x00BD2014$$

在 x32dbg 中跳转到 0x00BD2014，观察加载后的 IAT 表项。



(a) x32dbg 在 CPU 窗口跳转到加载后 IAT 地址

(b) x32dbg 内存窗口查看加载后 IAT 内容

图 15 加载后 IAT 表项被系统加载器改写为真实 API 地址

如图 15 所示，加载后的 0x00BD2014 处已经不再保存加载前文件中的 0x21C2。在 CPU 窗口中，x32dbg 对该位置进行符号解析，右侧显示为 GetDlgItemTextA；在内存窗口中，该地址附近显示为多个 4 字节一组的内存地址。这说明程序加载后，系统加载器已经根据导入表和 DLL 导出表，将 IAT 中原本的 Hint/Name RVA 改写为实际的 API 函数入口地址。

需要注意的是，图 15a 中 CPU 窗口会将 IAT 表项中的数据误按指令形式反汇编，因此出现 push、movsb、ja 等指令显示。这并不表示该位置是正常代码，而是因为 IAT 本质上是数据表。真正需要关注的是该地址所在位置和符号解析结果，即 0x00BD2014 对应 GetDlgItemTextA。

1.3 EPO 之指令 Patch

1.3.1 实验目标与修改思路

本实验以 Control.exe 为对象，完成 EPO 方式的指令 Patch。与直接修改程序入口点不同，EPO 方法不改变 AddressOfEntryPoint，而是在原程序正常执行路径中选择一条已有指令，将其改为跳转或调用寄生代码。寄生代码执行结束后再跳回原有执行流程，从而保持程序整体运行逻辑不被破坏。

本实验选择在 .text 节末尾空洞写入寄生代码，然后将入口点附近原本调用 GetModuleHandleA 的相对 CALL 指令修改为调用寄生代码。寄生代码执行后，再跳回原来的 GetModuleHandleA 导入跳板位置，使程序继续执行原逻辑。

1.3.2 定位 .text 节空洞

首先使用 PEView 打开 Control.exe，查看 IMAGE_SECTION_HEADER .text，记录 .text 节的关键字段。

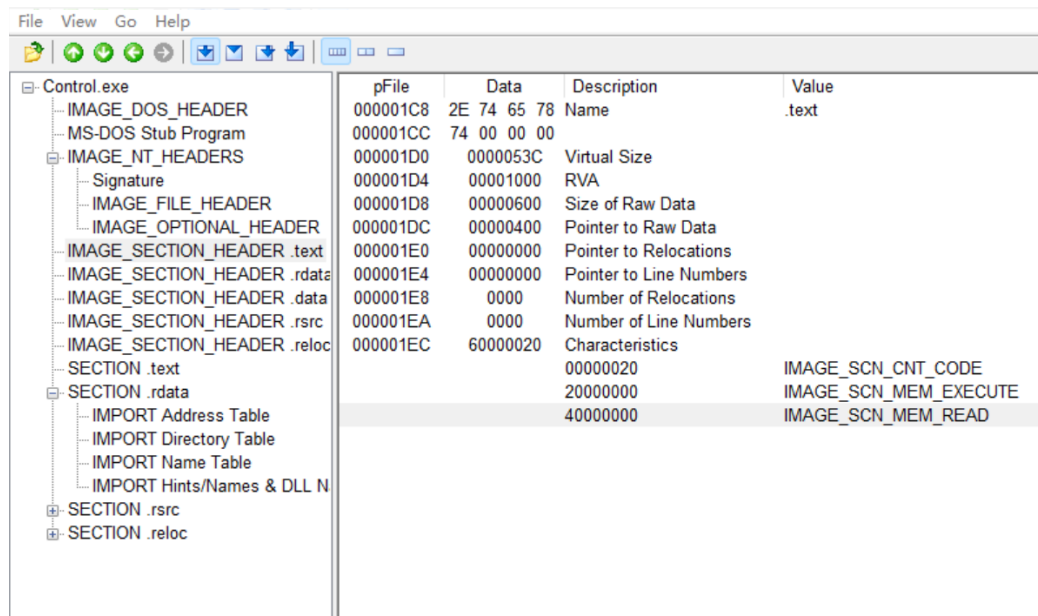


图 16 查看 .text 节原始字段

如图 16 所示，.text 节的原始字段为：

VirtualSize = 0x053C, RVA = 0x1000, SizeOfRawData = 0x0600,
PointerToRawData = 0x0400

因此，.text 节中已使用区域的末尾文件偏移为：

$$0x0400 + 0x053C = 0x093C$$

对应的 RVA 为：

$$0x1000 + 0x053C = 0x153C$$

同时，.text 节剩余空洞大小为：

$$0x0600 - 0x053C = 0x00C4$$

本实验写入的寄生代码长度为 0x16 字节，小于剩余空洞大小 0x00C4，因此可以将寄生代码写入 0x093C 处。

1.3.3 写入寄生代码

使用 ImHex 跳转到文件偏移 0x093C，写入寄生代码。寄生代码的主要作用是通过 `call-pop` 结构取得自身附近地址，将学号后 8 位 80904007 写入原先预留的 4 个 NOP 位置，然后跳回原程序执行流程。

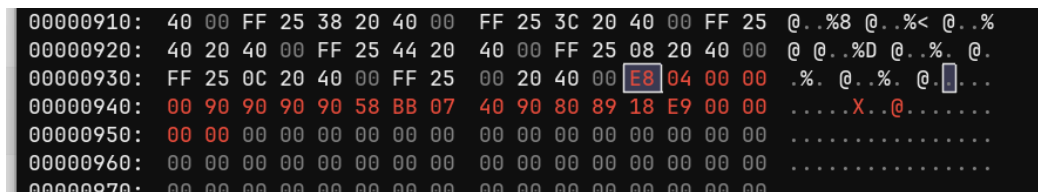


图 17 在 0x093C 处写入寄生代码

如图 17 所示，写入的寄生代码为：

```
E8 04 00 00 00 90 90 90 90 58 BB 07 40 90 80 89 18 E9 00 00 00 00
```

其中，BB 07 40 90 80 对应指令：

```
mov ebx, 0x80904007
```

由于 x86 中立即数按小端序存储，因此学号后 8 位 80904007 在文件中写作：

```
07 40 90 80
```

寄生代码最后的 E9 00 00 00 00 暂时作为占位，后续再根据需求跳回的位置重新计算相对偏移。

1.3.4 修改 .text 节表字段

写入寄生代码后，需要修改 .text 节的 VirtualSize，使新增代码能够被加载到内存中。原始 VirtualSize 为 0x053C，新增寄生代码长度为 0x16，因此新的 VirtualSize 为：

$$0x053C + 0x16 = 0x0552$$

同时，寄生代码中存在写入自身附近内存的操作，因此需要为 .text 节增加可写权限。原始节属性为：

```
Characteristics = 0x60000020
```

加入 IMAGE_SCN_MEM_WRITE = 0x80000000 后得到：

$$0x60000020 \mid 0x80000000 = 0xE0000020$$

地址	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
000001D0:	52	05	00	00	00	10	00	00	00	06	00	00	00	04	00	00	R.....
000001E0:	00	00	00	00	00	00	00	00	00	00	00	00	20	00	00	E0	
000001F0:	2E	72	64	61	74	61	00	00	F6	02	00	00	00	20	00	00	.rdata.....
00000200:	00	04	00	00	00	0A	00	00	00	00	00	00	00	00	00	00	
00000210:	00	00	00	00	40	00	00	40	2E	64	61	74	61	00	00	00	...@..@.data...
00000220:	10	00	00	00	00	30	00	00	00	00	00	00	00	00	00	00	0.....

图 18 修改 .text 节的 VirtualSize 和 Characteristics

如图 18 所示，文件偏移 0x1D0 处的 VirtualSize 被修改为：

52 05 00 00

即：

0x00000552

文件偏移 0x1EC 处的 Characteristics 被修改为：

20 00 00 E0

即：

0xE0000020

1.3.5 Patch 入口附近的 CALL 指令

原计划可以搜索导入跳板形式的 FF 25 指令，但这类指令中包含绝对地址，加载时可能受到重定位表影响。因此本实验最终选择入口点附近一条相对 CALL 指令作为 Patch 点。相对 CALL 使用相对偏移，不依赖固定绝对地址，修改后更稳定。

入口点附近原始指令位于文件偏移 0x0806，其原始机器码为：

E8 25 01 00 00

该指令对应调用 GetModuleHandleA 的导入跳板。该位置的 RVA 为：

$$0x0806 - 0x0400 + 0x1000 = 0x1406$$

寄生代码 RVA 已确定为 0x153C，因此相对 CALL 的偏移计算如下：

$$0x153C - (0x1406 + 5) = 0x0131$$

所以将原始机器码修改为：

E8 31 01 00 00

地址	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
00000800:	C9	C2	10	00	6A	00	E8	31	01	00	00	A3	00	30	40	00	j.1....0@.
00000810:	6A	00	68	00	10	40	00	6A	00	6A	01	FF	35	00	30	40	j.h..@.j.j..5.0@
00000820:	00	E8	9E	00	00	00	6A	00	E8	FD	00	00	00	90	90	90	j.....
00000830:	90	90	90	90	90	90	90	90	90	90	90	90	90	90	90	90	
00000840:	90	90	90	90	90	90	90	90	90	90	90	90	90	90	90	90	
00000850:	90	90	90	90	90	90	90	90	90	90	90	90	90	90	90	90	

图 19 将入口附近原有 CALL 指令 Patch 为调用寄生代码

如图 19 所示，文件偏移 0x0806 处已经修改为 E8 31 01 00 00，表示程序执行到该处时会调用 RVA 0x153C 处的寄生代码。

1.3.6 修改寄生代码尾部跳转

寄生代码执行结束后，需要跳回原来的 GetModuleHandleA 导入跳板，使程序能够继续执行原流程。根据调试结果，原 GetModuleHandleA 导入跳板对应 RVA 为 0x1530。

寄生代码尾部 JMP 指令位于文件偏移 0x094D，其 RVA 为：

$$0x094D - 0x0400 + 0x1000 = 0x154D$$

E9 指令长度为 5 字节，因此下一条指令 RVA 为：

$$0x154D + 5 = 0x1552$$

跳回目标 RVA 为 0x1530，所以相对偏移为：

$$0x1530 - 0x1552 = -0x22$$

将 -0x22 转换为 32 位补码并按小端序写入，得到：

DE FF FF FF

因此寄生代码尾部由：

E9 00 00 00 00

修改为：

E9 DE FF FF FF

```

00000920: 40 20 40 00 FF 25 44 20 40 00 FF 25 08 20 40 00 @ @.%.D @.%. @.
00000930: FF 25 0C 20 40 00 FF 25 00 20 40 00 E8 04 00 00 .%. @.%. @.
00000940: 00 90 90 90 90 58 BB 07 40 90 80 89 18 E9 DE FF .....X. @.
00000950: FF FF 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000960: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

图 20 修改寄生代码尾部 JMP，使其跳回原导入跳板

如图 20 所示，文件偏移 0x094D 处已经修改为 E9 DE FF FF FF，表示寄生代码执行完毕后跳回原来的 GetModuleHandleA 调用链。

1.3.7 动态调试验证

修改完成后，使用 x32dbg 打开修改后的 Control.exe。程序入口点地址显示为 0x00CC1404，对应原入口点 RVA 0x1404，因此本次加载基址为：

$$0x00CC1404 - 0x1404 = 0x00CC0000$$

入口附近被 Patch 的指令 RVA 为 0x1406，因此其实际内存地址为：

$$0x00CC0000 + 0x1406 = 0x00CC1406$$

寄生代码 RVA 为 0x153C，因此其实际内存地址为：

$$0x00CC0000 + 0x153C = 0x00CC153C$$

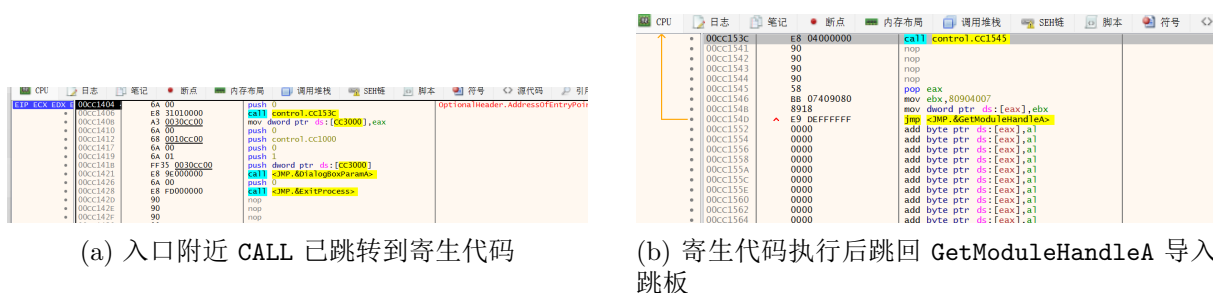


图 21 x32dbg 中验证 EPO Patch 后的执行链

如图 21 所示，入口附近地址 0x00CC1406 处的指令已经变为：

call control.CC153C

说明原程序执行到该位置时，会先进入寄生代码。跳转到 0x00CC153C 后，可以看到寄生代码的完整指令序列：

```

call control.CC1545

nop nop nop nop

pop eax

mov ebx, 80904007

mov dword ptr ds:[eax], ebx

jmp <JMP.&GetModuleHandleA>

```

这说明程序执行流已经由原入口附近的 CALL 指令进入寄生代码，寄生代码写入学号后 8 位 80904007 后，又通过尾部 JMP 跳回原来的 GetModuleHandleA 导入跳板。由此验证，本实验在不修改 AddressOfEntryPoint 的情况下，完成了基于 EPO 的指令 Patch，并保持了原程序执行流程的连续性。

六、实验结论：

本次实验完成了对 DLL 导出表、DLL 导入表和 EPO 指令 Patch 三个部分的分析与验证。在导出表实验中，通过 user32.dll 的 EXPORT Table 成功定位到 IMAGE_EXPORT_DIRECTORY，并根据函数名指针表、序号表和函数地址表，查找到学号后两位 07 对应的导出函数 AnimateWindow。实验结果表明，AnimateWindow 的函数地址表索引为 0x0006，导出序号为 7，函数入口 RVA 为 0x00012156，对应文件偏移为 0x11556。在 ImHex 中观察到该位置的机器码 8B FF 55 8B EC，验证了导出函数地址查找过程正确。

在导入表实验中，通过 Control.exe 的数据目录定位到 IMPORT Table RVA = 0x20C4，并根据 .rdata 节表字段计算出导入表文件偏移为 0x0AC4。通过观察 IMPORT Directory Table，确认程序导入了 user32.dll、kernel32.dll 和 gdi32.dll。进一步对比 IMPORT Name Table 和 IMPORT Address Table 可知，程序加载前 IAT 表项保存的是 Hint/Name RVA，例如 GetDlgItemTextA 对应的表项为 0x21C2；通过 ImHex 跳转到 0x0BC2 后，可以看到 Hint = 0x00FC 和函数名字符串 GetDlgItemTextA。在 x32dbg 中观察加载后的 IAT，可以确认系统加载器已经将 IAT 表项改写为真实 API 函数地址，验证了 PE 导入表的动态解析机制。

在 EPO 指令 Patch 实验中，首先根据 Control.exe 的 .text 节表字段计算出节末尾空洞文件偏移为 0x093C，对应 RVA 为 0x153C，并成功写入长度为 0x16 字节的寄生代码。随后将 .text 节的 VirtualSize 从 0x053C 修改为 0x0552，将 Characteristics 从 0x60000020 修改为 0xE0000020，保证新增代码能够被加载并具有写权限。最后将入口附近文件偏移 0x0806 处的 CALL 指令由 E8 25 01 00 00 修改为 E8 31 01 00 00，使程序执行流进入寄生代码；再将寄生代码尾部 JMP 修改为 E9 DE FF FF FF，使其跳回原 GetModuleHandleA 导入跳板。x32dbg 调试结果表明，程序执行流能够按照“入

口附近 CALL → 寄生代码 → 原导入跳板”的顺序运行，说明 EPO 指令 Patch 实验成功完成。

综合三个实验可以得出结论：PE 文件中的导出表、导入表和指令执行流之间存在明确的结构关系。导出表解决 DLL 内部“函数名到函数地址”的映射问题，导入表解决 EXE 加载时“外部函数引用到真实 API 地址”的解析问题，而 EPO 指令 Patch 则是在理解 PE 节结构、RVA 换算、指令编码和加载机制基础上，对程序执行流进行受控修改。通过本次实验，可以较完整地理解 PE 文件从静态结构到动态执行之间的联系。

七、总结及心得体会：

本次实验相比前面的文件结构分析实验，更强调“表结构、文件字节和运行时行为”之间的对应关系。仅在 PEView 中看到表项还不够，还需要使用 ImHex 回到文件原始字节中验证字段是否真实存在，再使用 x32dbg 在程序运行后观察这些字段如何被系统加载器解释和改写。通过这种静态与动态结合的方式，能够更清楚地理解 PE 文件不是孤立的十六进制数据，而是一套会被系统加载器按规则解析并执行的结构化文件。

在 DLL 导出表实验中，我对函数名、序号和函数地址之间的关系有了更直观的认识。导出函数的查找并不是简单搜索字符串，而是需要经过函数名指针表、序号表和函数地址表的多级映射。通过 AnimateWindow 的查找过程可以看到，函数名字符串、导出序号和函数入口 RVA 分别存放在不同表中，只有把这些表联系起来，才能准确定位函数入口。

在 DLL 导入表实验中，我进一步理解了 IAT 的作用。程序文件在磁盘上无法提前知道系统 DLL 被加载到哪个地址，因此加载前的 IAT 只能保存指向函数名结构的 RVA；程序运行时，加载器再根据 DLL 导出表解析真实函数地址并写回 IAT。这个过程解释了为什么同一个 IAT 表项在文件中和内存中会呈现不同内容，也说明分析 PE 文件时不能只看静态文件，还必须考虑加载后的内存状态。

在 EPO 指令 Patch 实验中，最关键的体会是补丁修改必须服从指令编码和加载机制。最开始尝试修改导入跳板形式的 FF 25 指令时，由于该位置涉及绝对地址和重定位，调试时会出现与文件中原始字节不一致的情况。后来改为 Patch 入口附近的相对 CALL 指令，问题才得到解决。这说明在进行执行流修改时，不能只看某条指令表面上能不能跳转，还要判断它是否会受重定位、加载基址和指令长度影响。

通过本次实验，我对 PE 文件分析的基本思路更加清晰：先用结构化工具确定字段含义，再用十六进制编辑器验证和修改原始字节，最后用调试器验证运行时效果。这个过程也说明，PE 文件分析的重点不是单纯记忆某个字段名称，而是能够把字段值、地址换算、指令编码和程序行为连接起来，形成完整的证据链。

报告评分：

指导教师签字：